Banking Client-Server Communication Protocol

Table of Contents

1. Overview & Transport
2. Message Framing
3. Message Envelopes
4. Connection & Session Lifecycle
5. API Actions Reference (1–13)
6. Protocol Design Rationale
7. Client Implementation Notes

1. Overview & Transport

Transport Layer: Communication occurs over a persistent TCP stream socket. The default binding host is `127.0.0.1` (or `0.0.0.0`) on port `5050`.

Interaction Model: Synchronous request-response Remote Procedure Call (RPC) model over a stateful TCP connection. Clients issue a single request and must wait for the server response before sending subsequent messages. Request pipelining is not supported.

2. Message Framing

Messages are formatted using length-prefixed binary frames to guarantee clear message boundaries across stream segmentations.

Header Structure: A 4-byte unsigned integer in big-endian network byte order specifying payload length `N`.

Payload Structure: Exactly `N` bytes of UTF-8 encoded JSON text immediately following the 4-byte header.

Payload Size Limits: Minimum payload size is 1 byte. Maximum payload size is 1,048,576 bytes (1 MiB).

Stream Reading Procedure: Receiving endpoints read exactly 4 header bytes, decode the integer length `N`, validate that `1 <= N <= 1,048,576`, and then read exactly `N` bytes of payload before parsing JSON.

3. Message Envelopes

Client Request Envelope

Every client message MUST be a JSON object with integer `action` (1 to 13) and object `payload`. Actions without parameters must send `"payload": {}`.

```
JSON
{"action": 4, "payload": {"target": "A1B2C3D4E5", "amount": "25.50"}}
```

Server Success Response Envelope (status == 1)

```
JSON
{"status": 1, "data": {"balance": "474.50"}}
```

Server Error Response Envelope (status == 2)

```
JSON
{"status": 2, "data": {"error": "Insufficient funds"}}
```

4. Connection & Session Lifecycle

- **Unauthenticated State:** Following TCP connection establishment, sockets are unauthenticated. Only `REGISTER_USER` (1) and `LOGIN` (2) are permitted. Protected actions return an error.
- **Authenticated State:** Successfully executing `LOGIN` (2) binds the user session directly to the open TCP socket. Attempting a second login on an already authenticated connection is rejected with an application error.
- **Disconnection & Logout:** Executing `LOGOUT` (9) unbinds the session. Closing or dropping the TCP socket automatically clears the server session mapping.
- **Error Handling:** Severe errors (invalid frame lengths, malformed JSON, truncated frames, or unknown action codes) indicate stream corruption; the server responds with an error frame and immediately terminates the TCP connection. Application and business logic errors (such as invalid credentials or insufficient funds) return a status 2 error payload while leaving the TCP connection open.

5. API Actions Reference (1–13)

Action Summary Table

Code	Enum Name	Auth	Request Payload Fields	Success Response Data
1	REGISTER_USER	No	username, password	userid
2	LOGIN	No	user_id, password	status, user
3	GET_BALANCE	Yes	{}	balance
4	TRANSFER	Yes	target, amount	balance
5	STANDING_ORDER	Yes	target, amount, count	so_id
6	TAKE_LOAN	Yes	amount	loan_id, balance
7	CANCEL_STANDING_ORDER	Yes	so_id	status
8	GET_LOANS	Yes	{}	loans
9	LOGOUT	Yes	{}	status
10	DEPOSIT	Yes	amount	balance
11	WITHDRAW	Yes	amount	balance
12	GET_HISTORY	Yes	{} or from_ts, to_ts	history
13	GET_STANDING_ORDERS	Yes	{}	standing_orders

Action Specifications

1: REGISTER_USER

Creates a new user account. Payload accepts `username` (3-20 alphanumeric/underscore characters) and `password` (min 4 characters). Returns a 10-character alphanumeric `userid`.

```
JSON
{"action": 1, "payload": {"username": "alice_99", "password":
"SecretPassword123"}}
{"status": 1, "data": {"userid": "K9B2X7W4M1"}}
```

2: LOGIN

Authenticates a user session on the current socket. Payload accepts `user_id` (10 chars) and `password`. Returns login status and display username.

```
JSON
{"action": 2, "payload": {"user_id": "K9B2X7W4M1", "password":
"SecretPassword123"}}
{"status": 1, "data": {"status": 1, "user": "alice_99"}}
```

3: GET_BALANCE

Retrieves the current available balance for the authenticated account as a two-decimal string.

```
JSON
{"action": 3, "payload": {}}
{"status": 1, "data": {"balance": "1050.75"}}
```

4: TRANSFER

Transfers funds to a target user account. Requires recipient `target` user ID and positive decimal `amount` string.

```
JSON
{"action": 4, "payload": {"target": "P5M8W1Q3Z9", "amount": "125.50"}}
{"status": 1, "data": {"balance": "925.25"}}
```

5: STANDING_ORDER

Schedules recurring automatic payments. Payload requires `target`, decimal string `amount`, and string digit `count` ("0" or "" for indefinite, N > 0 for fixed count).

```
JSON
{"action": 5, "payload": {"target": "P5M8W1Q3Z9", "amount": "50.00", "count":
"12"}}
{"status": 1, "data": {"so_id": 104}}
```

6: TAKE_LOAN

Issues a bank loan credited immediately to balance. Payload requires positive decimal string `amount`. Returns loan ID and updated balance.

```
JSON
{"action": 6, "payload": {"amount": "500.00"}}
{"status": 1, "data": {"loan_id": 12, "balance": "1425.25"}}
```

7: CANCEL_STANDING_ORDER

Cancels an existing standing order. Payload accepts `so_id` (integer or string of digits).

```
JSON
{"action": 7, "payload": {"so_id": 104}}
{"status": 1, "data": {"status": 1}}
```

8: GET_LOANS

Retrieves active and past loans as positional arrays containing: [loan_id, principal, repay_amount, taken_on, due_on].

```
JSON
{"action": 8, "payload": {}}
{"status": 1, "data": {"loans": [[12, "500.00", "525.00", "2025-01-01",
"2025-01-31"]]}}
```

9: LOGOUT

Logs out the current session on the socket.

```
JSON
{"action": 9, "payload": {}}
{"status": 1, "data": {"status": 1}}
```

Decimal Strings for Monetary Values: Prevents floating-point rounding inaccuracies in financial transactions.

Positional Arrays for Bulk Lists: Reduces payload sizes for tabular lists (history, loans, standing orders) by avoiding repeated key names in multi-row datasets.

Connection-Bound Sessions: Binds authenticated state directly to socket connection lifetimes, eliminating per-request token overhead and ensuring instant cleanup on disconnect.

Strict Field Validation: Enforces exact field requirements to reject unexpected keys or parameter tampering.

- *Mandatory Envelope Rule:* For actions taking no parameters, the client MUST transmit an empty JSON object `{}`. Omitting the `payload` key or providing `null` results in a structural rejection.

7. Client Implementation Notes

ID Field Naming Asymmetry: Registration response returns `"userid"` (no underscore), whereas Login request expects `"user_id"` (with underscore).

Standing Order Count Formatting: `STANDING_ORDER` (5) requires `count` to be transmitted as a string of digits (e.g. `"12"` or `"0"`), not an integer.

```
None
{
  "status": 1,
  "data": {
    "balance": "474.50"
  }
}
```

Strict Type Enforcement: Never send JSON booleans (`true` / `false`) for integer or timestamp fields; the server explicitly checks types and rejects booleans.

```
None
{
  "status": 2,
  "data": {
    "error": "Insufficient funds"
  }
}
```

Mandatory Payload Key: Requests without arguments must explicitly include an empty object payload (`"payload": {}`).